



Symbiosis Institute of Technology, Pune

CSE- Academic Year 2026-27

Java Programming – Lab Batch 2025-29

Lab Assignment No.: 11

Name of Student	Akshit Gawade
PRN No.	25070122029
Batch	2025-2029
Class	A1
Academic Year & Semester	Second Year Semester III
Date of Submission	
Title of Assignment:	Write a Java program to demonstrate the implementation of interfaces.
Problem Statement	Write a Java program to demonstrate the implementation of interfaces.
Learning Objectives	To implement interfaces for achieving abstraction and multiple inheritance of type.
Theory	An interface specifies abstract methods that implementing classes must define. It provides abstraction and supports multiple inheritance of type.
Program Code	<pre>interface Animal { void sound(); } class Owl implements Animal { public void sound() { System.out.println("Owl hoots."); } } public class Main { public static void main (String[] args) {</pre>

	<pre> Animal a = new Owl(); a.sound(); } } </pre>
Output Screenshots	<pre> ● PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 11> & 'c:\Program Files\Eclipse Adoptium\jdk-24.0 .2.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Nilima\AppData\ Roaming\Code\User\workspaceStorage\f4c886e1461e070b23ae89e23eaf88ee\redhat.java\jdt_ws\LAB EXP 11_cc8 0a228\bin' 'Main' Owl hoots. ○ PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 11> </pre>
Exercise Qns	Q1. Create an interface Printable and implement it in Student and Employee classes. <pre> interface Print { void print(); } class Student implements Print { String name; int prn; Student(String name, int prn) { this.name = name; this.prn = prn; } public void print() { System.out.println("Name of Student: " + name); System.out.println("PRN: " + prn); } } class Employee implements Print { String name; int empId; Employee(String name, int empId) { this.name = name; this.empId = empId; } } </pre>

```
public void print() {  
    System.out.println("Employee Name: " + name);  
    System.out.println("Employee ID: " + empId);  
}  
}
```

```
public class Exmp1 {  
    public static void main(String[] args) {  
        Print s = new Student("Akshit", 29);  
        Print e = new Employee("Agam", 25);
```

```
        s.print();  
        System.out.println();  
        e.print();  
    }  
}
```

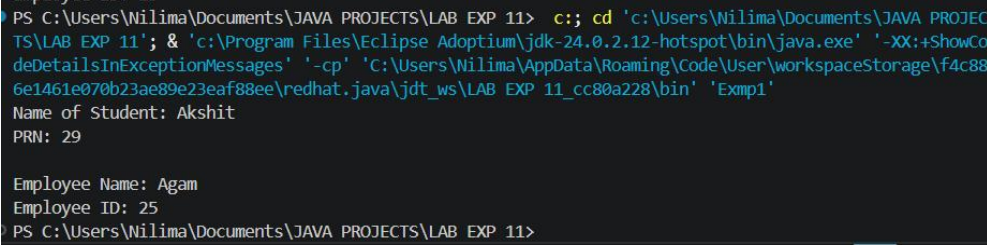
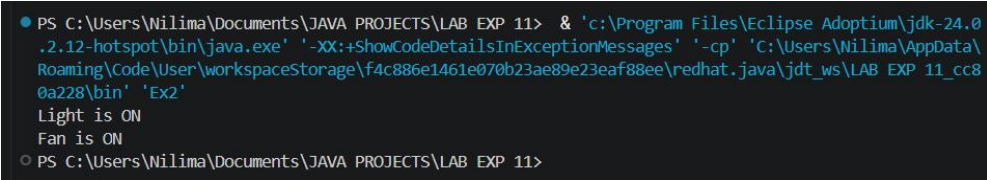
Q2. Create an interface Switchable with a method turnOn(). Implement the interface in Light and Fan classes to display the device status.

```
interface Switchable {  
    void turnOn();  
}  
  
class Light implements Switchable {  
    public void turnOn() {  
        System.out.println("Light is ON");  
    }  
}
```

```
class Fan implements Switchable {  
    public void turnOn() {  
        System.out.println("Fan is ON");  
    }  
}
```

```
public class Ex2 {  
    public static void main(String[] args) {  
        Switchable light = new Light();  
        Switchable fan = new Fan();
```

```
        light.turnOn();  
        fan.turnOn();  
    }  
}
```

	<pre>} </pre>
Output Screenshots	1.  <pre>PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 11> c::; cd 'c:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 11'; & 'c:\Program Files\Eclipse Adoptium\jdk-24.0.2.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Nilima\AppData\Roaming\Code\User\workspaceStorage\f4c886e1461e070b23ae89e23eaf88ee\redhat.java\jdt_ws\LAB EXP 11_cc80a228\bin' 'Exmp1' Name of Student: Akshit PRN: 29 Employee Name: Agam Employee ID: 25 PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 11></pre> 2.  <pre>PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 11> & 'c:\Program Files\Eclipse Adoptium\jdk-24.0.2.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Nilima\AppData\Roaming\Code\User\workspaceStorage\f4c886e1461e070b23ae89e23eaf88ee\redhat.java\jdt_ws\LAB EXP 11_cc80a228\bin' 'Ex2' Light is ON Fan is ON PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 11></pre>
Conclusion	This assignment helped in understanding the concept and implementation of interfaces in Java. It demonstrated how interfaces provide complete abstraction by defining a common set of methods that different classes can implement according to their specific requirements. The implementation improved my understanding of object-oriented programming concepts such as abstraction, polymorphism, and loose coupling, resulting in flexible and reusable code. These concepts are widely used in developing desktop applications, web applications, Android applications, enterprise software, banking systems, and other real-world Java applications that require modular, scalable, and maintainable software design.